

1. Resumen

Frost Puno es un sistema inteligente orientado a la predicción del riesgo de heladas en comunidades altoandinas de la región Puno, con énfasis en el apoyo a productores agrícolas y productores de chuño. El proyecto aborda una problemática regional relevante: la exposición de cultivos, ganado y actividades productivas tradicionales a descensos críticos de temperatura, especialmente en zonas de alta altitud donde las decisiones suelen basarse en experiencia empírica y no siempre en información climática integrada.

La solución implementada corresponde a un Producto Mínimo Viable (MVP) que integra datos abiertos meteorológicos, territoriales y agropecuarios. Open-Meteo se utiliza como fuente climática principal para obtener variables horarias por coordenadas; INEI se emplea como fuente territorial y censal mediante una semilla curada documentada para distritos de Puno; SENAMHI se considera como fuente oficial peruana para validación climática en una segunda fase; y MIDAGRI/SIEA se plantea como fuente complementaria para enriquecer el contexto agrícola.

Desde el enfoque de Aprendizaje de Máquina, el sistema construye un dataset tabular, genera variables predictoras, etiqueta inicialmente el riesgo de helada mediante reglas térmicas y entrena modelos supervisados de clasificación multiclase. Se comparan LogisticRegression, DecisionTreeClassifier y RandomForestClassifier, seleccionando RandomForestClassifier como modelo principal. La métrica principal es f1-score macro, adecuada para un problema con tres clases y posible desbalance.

Desde Computación Paralela y Distribuida, el proyecto evidencia procesamiento concurrente por distritos durante la ingesta climática, separación modular de responsabilidades, backend FastAPI, base de datos Supabase preparada, cliente Flutter y flujos CI/CD con GitHub Actions. El sistema no pretende reemplazar alertas oficiales, sino demostrar una arquitectura académica funcional, extensible y verificable para predicción de riesgo agroclimático.

2. Introducción

La región Puno presenta condiciones climáticas particulares debido a su altitud, geografía altoandina y variabilidad térmica. Las heladas constituyen un fenómeno recurrente que afecta a comunidades rurales, productores de papa, ganaderos y familias dedicadas a la producción tradicional de chuño. En este contexto, la anticipación del riesgo de helada es un factor importante para la toma de decisiones agrícolas, la protección de cultivos sensibles y la planificación de actividades productivas.

Tradicionalmente, muchos productores interpretan señales del entorno y toman decisiones a partir de conocimiento empírico acumulado. Dicho conocimiento es valioso, pero puede fortalecerse mediante herramientas computacionales que integren datos meteorológicos, ubicación geográfica, altitud, información territorial y aprendizaje automático.

Frost Puno surge como una propuesta académica y tecnológica para demostrar que es posible construir un sistema distribuido, modular y basado en datos abiertos capaz de estimar el riesgo de helada en niveles bajo, medio y alto. El proyecto combina una aplicación Flutter, un backend FastAPI, un pipeline

de Machine Learning en Python con Scikit-learn, una base de datos Supabase preparada para persistencia y workflows de GitHub Actions para validación continua.

El propósito del MVP no es emitir alertas oficiales ni reemplazar a instituciones especializadas como SENAMHI, sino construir una base tecnológica verificable para investigación, exposición universitaria y mejora progresiva del modelo.

3. Planteamiento del problema

3.1. Problema general

¿Cómo desarrollar un sistema inteligente distribuido que integre datos abiertos meteorológicos, territoriales y agropecuarios para estimar el riesgo de heladas en comunidades altoandinas de Puno y apoyar la toma de decisiones relacionadas con cultivos y producción de chuño?

3.2. Problemas específicos

1. ¿Cómo integrar datos climáticos por coordenadas con información territorial y censal de distritos de Puno?
2. ¿Cómo construir un dataset inicial para clasificar el riesgo de helada en bajo, medio y alto?
3. ¿Qué modelo supervisado tabular resulta adecuado para un MVP académico con recursos computacionales limitados?
4. ¿Cómo evidenciar procesamiento paralelo y arquitectura distribuida sin desplegar microservicios costosos?
5. ¿Cómo exponer predicciones mediante una API modular consumida por una aplicación Flutter?
6. ¿Cómo registrar, evaluar y gobernar versiones del modelo mediante CI/CD y quality gates?

4. Justificación

4.1. Justificación social

Las heladas afectan directamente a familias rurales, agricultores, ganaderos y productores de chuño. Una herramienta que permita consultar el riesgo de helada de forma sencilla puede contribuir a mejorar la planificación de labores agrícolas y la protección de cultivos sensibles. Aunque el MVP no reemplaza sistemas oficiales, sirve como base para acercar información técnica a usuarios no especializados.

4.2. Justificación tecnológica

El proyecto demuestra la integración de tecnologías modernas: Flutter para frontend móvil/web, FastAPI para backend, Scikit-learn para aprendizaje supervisado, Supabase para persistencia y GitHub Actions para automatización. La arquitectura modular permite escalar hacia servicios especializados en fases posteriores.

4.3. Justificación académica

Frost Puno articula dos áreas centrales de la Ingeniería de Sistemas: Computación Paralela y Distribuida, y Aprendizaje de Máquina. El sistema permite demostrar ingesta concurrente, separación de responsabilidades, validación automatizada, entrenamiento de modelos, evaluación de métricas y ciclo de vida ML.

4.4. Justificación regional

Puno requiere soluciones contextualizadas a su geografía, altitud y actividades productivas. El uso de datos abiertos y variables territoriales facilita construir una herramienta vinculada a la realidad regional, especialmente para distritos y centros poblados altoandinos.

5. Objetivos

5.1. Objetivo general

Desarrollar un sistema inteligente distribuido para predecir el riesgo de heladas en comunidades altoandinas de Puno mediante aprendizaje supervisado, integración de datos abiertos y una arquitectura modular orientada a servicios.

5.2. Objetivos específicos

7. Integrar datos climáticos desde Open-Meteo con datos territoriales compatibles con INEI.
8. Construir un dataset inicial con variables meteorológicas, geográficas y poblacionales.
9. Entrenar y comparar modelos supervisados para clasificar el riesgo de helada.
10. Implementar un backend FastAPI modular con endpoints de predicción e información del modelo.
11. Preparar una estructura Supabase con tablas, políticas RLS y repositorio opcional.
12. Desarrollar una aplicación Flutter modular inspirada en el diseño visual de Stitch.
13. Implementar workflows de CI/CD para pruebas, validación de datos, entrenamiento y quality gate.
14. Documentar limitaciones, resultados y fases futuras del sistema.

6. Alcance del proyecto

El alcance actual corresponde a un MVP académico funcional. Incluye:

- Pipeline ML para ingesta, features, validación, entrenamiento, evaluación y registry.
- Uso de Open-Meteo como fuente climática principal.
- Semilla territorial curada compatible con estructura INEI para distritos seleccionados de Puno.
- Modelo RandomForestClassifier registrado.
- Backend FastAPI con endpoints mínimos funcionales.
- Estructura Supabase con migraciones, seed y políticas RLS.
- Aplicación Flutter Web/móvil con pantallas principales.

- CI/CD con GitHub Actions para validación del backend, datos y modelo.

Queda fuera del MVP:

- Integración completa automatizada de INEI EstaDist, Microdatos o CENAGRO.
- Validación oficial completa con estaciones SENAMHI.
- Alertas push reales.
- Mapa interactivo.
- Modo offline.
- Despliegue productivo completo.
- Validación con productores en campo.

7. Marco teórico

7.1. Heladas en comunidades altoandinas

Las heladas ocurren cuando la temperatura desciende hasta niveles que pueden afectar tejidos vegetales, agua superficial, suelos y actividades agropecuarias. En zonas altoandinas, la altitud, baja humedad nocturna, cielos despejados y vientos pueden intensificar el enfriamiento. En Puno, estos eventos tienen impacto sobre cultivos, pastos, animales y seguridad alimentaria.

7.2. Producción de chuño

El chuño es un producto tradicional obtenido mediante procesos de congelación y deshidratación de tubérculos, principalmente papa. Las bajas temperaturas nocturnas pueden ser favorables para su producción cuando se combinan con condiciones de baja precipitación y exposición adecuada. Por ello, un sistema que identifique condiciones de helada puede apoyar tanto la protección de cultivos como la planificación de producción de chuño.

7.3. Aprendizaje supervisado

El aprendizaje supervisado utiliza ejemplos etiquetados para entrenar modelos capaces de predecir una variable objetivo. En Frost Puno, la variable objetivo es `riesgo\_helada`, con tres clases: bajo, medio y alto. Las etiquetas iniciales se generan mediante reglas basadas en temperatura mínima diaria y horas bajo cero.

7.4. Random Forest

Random Forest es un método de ensamble que combina múltiples árboles de decisión entrenados sobre subconjuntos de datos y variables. Su ventaja principal es que suele ofrecer buen desempeño en datos tabulares, tolerancia a relaciones no lineales y menor tendencia al sobreajuste respecto a un árbol individual. En el MVP se eligió por su equilibrio entre precisión, interpretabilidad práctica y costo computacional razonable.

7.5. Clasificación multiclase

La clasificación multiclase consiste en asignar cada registro a una de varias categorías posibles. En este caso, las clases son `bajo`, `medio` y `alto`. Se usa f1-score macro porque calcula el promedio del desempeño por clase sin ponderar por frecuencia, lo cual es útil cuando puede existir desbalance.

7.6. Computación paralela

La computación paralela permite ejecutar múltiples tareas al mismo tiempo para reducir tiempos de procesamiento. En Frost Puno se evidencia mediante la descarga concurrente de clima por distrito usando `max-workers`, lo que permite consultar Open-Meteo para varias ubicaciones de forma simultánea.

7.7. Sistemas distribuidos

Un sistema distribuido separa responsabilidades entre componentes que interactúan mediante interfaces. Frost Puno distribuye funciones entre app Flutter, backend FastAPI, pipeline ML, Supabase y GitHub Actions. Aunque el MVP usa un backend modular monolítico, su diseño facilita migrar a microservicios.

7.8. CI/CD

La integración y entrega continua permiten automatizar pruebas, validación, entrenamiento y control de calidad. En este proyecto, GitHub Actions valida backend, datos y modelos para reducir errores manuales y asegurar reproducibilidad.

7.9. Ciclo de vida de modelos ML

El ciclo de vida ML comprende ingesta de datos, preparación, entrenamiento, evaluación, registro, monitoreo y mejora continua. Frost Puno implementa una versión inicial de este ciclo mediante scripts de pipeline, registry y quality gate.

8. Fuentes de datos

8.1. Open-Meteo

Open-Meteo es la fuente climática principal del MVP. Permite obtener datos históricos por coordenadas mediante su Historical Weather API. En el pipeline se consultan variables horarias como temperatura, humedad relativa, sensación térmica, punto de rocío, precipitación, nubosidad y velocidad del viento.

8.2. INEI

INEI se usa como fuente territorial, censal y agropecuaria de referencia. En el MVP se utiliza `data/external/inei\_puno\_districts.csv`, una semilla curada inicial compatible con estructura distrital de INEI. Esta semilla no representa una extracción completa automática desde INEI; debe reemplazarse o validarse con exportaciones oficiales de EstaDist, CPV 2017, Microdatos o CENAGRO en fases futuras.

8.3. SENAMHI

SENAMHI se considera fuente oficial peruana para validación climática y fases futuras. Su información de estaciones, avisos meteorológicos y datos hidrometeorológicos permitiría contrastar las predicciones generadas por el modelo y mejorar las etiquetas iniciales.

8.4. MIDAGRI/SIEA

MIDAGRI/SIEA se plantea como fuente agrícola complementaria para incorporar producción, superficie, rendimiento y cultivos relevantes. En el MVP se documenta su uso futuro para enriquecer recomendaciones y ponderar exposición productiva.

8.5. Limitaciones del dataset inicial

El dataset inicial utiliza etiquetas derivadas de reglas térmicas. Esto permite entrenar un modelo base, pero no equivale a observaciones oficiales de daño agrícola. Asimismo, la semilla territorial debe ser reemplazada por datos oficiales completos antes de un uso productivo.

9. Arquitectura del sistema

Frost Puno se organiza en componentes independientes:

- **Flutter:** interfaz móvil/web para consulta, resultado, historial, fuentes y modelo.
- **FastAPI:** backend que expone endpoints REST, valida datos con Pydantic y ejecuta predicciones.
- **Supabase:** base de datos PostgreSQL preparada para persistir predicciones, ubicaciones, modelos y logs.
- **ML pipeline:** scripts Python para ingesta, features, entrenamiento, evaluación y registry.
- **GitHub Actions:** automatización de pruebas, validación de datos, entrenamiento y quality gate.

9.1. Flujo de datos

15. El pipeline ingiere ubicaciones y clima.
16. Se construye el dataset de entrenamiento.
17. Se entrena y registra el mejor modelo.
18. FastAPI carga el modelo desde el registry.
19. Flutter envía un JSON de consulta a FastAPI.
20. FastAPI predice el riesgo y devuelve recomendación.
21. Si Supabase está habilitado, se registra la predicción en `frost\_predictions`.

9.2. Backend modular monolítico

Para el MVP se eligió un backend modular monolítico en lugar de microservicios reales porque:

- reduce complejidad de despliegue;
- evita costos de infraestructura;
- mantiene el proyecto ejecutable en una laptop;

- permite demostrar separación de responsabilidades mediante carpetas, servicios y repositorios;
- facilita migrar a microservicios si el sistema crece.

10. Computación Paralela y Distribuida

El proyecto evidencia conceptos de computación paralela y distribuida en distintos niveles:

10.1. Procesamiento por distritos y centros poblados

El diseño contempla procesar datos climáticos por distrito o centro poblado. En el MVP se trabaja con distritos seleccionados de Puno.

10.2. Uso de max-workers

El script `ingest_weather_open_meteo.py` permite configurar `--max-workers`, ejecutando múltiples solicitudes concurrentes hacia Open-Meteo. Esto reduce el tiempo de ingesta cuando se procesan varias ubicaciones.

10.3. Consultas concurrentes a Open-Meteo

Cada ubicación puede consultarse de forma independiente, lo cual se ajusta naturalmente a un patrón paralelo. Las respuestas se integran luego en un dataset común.

10.4. Jobs independientes en CI/CD

Los workflows de GitHub Actions separan responsabilidades:

- pruebas backend;
- validación de datos;
- entrenamiento ML;
- quality gate.

Esto representa una distribución lógica de tareas de mantenimiento del sistema.

10.5. Escalabilidad futura

En fases posteriores, los módulos podrían separarse en servicios independientes:

- servicio climático;
- servicio territorial;
- servicio ML;
- servicio de alertas;
- servicio de historial;
- servicio de entrenamiento.

11. Aprendizaje de Máquina

11.1. Tipo de problema

El problema se formula como clasificación supervisada multiclase. Cada registro representa condiciones climáticas y territoriales para una ubicación y tiempo específico.

11.2. Variable objetivo

La variable objetivo es:

`riesgo_helada`

11.3. Clases

- ``bajo``
- ``medio``
- ``alto``

11.4. Variables predictoras

Entre las variables usadas se incluyen:

- `latitud;`
- `longitud;`
- `altitud estimada;`
- `población total;`
- `población rural;`
- `porcentaje rural;`
- `temperatura;`
- `humedad relativa;`
- `sensación térmica;`
- `punto de rocío;`
- `precipitación;`
- `nubosidad;`
- `viento;`
- `mes;`
- `hora;`
- `temperatura mínima diaria;`
- `horas bajo cero.`

11.5. Modelos comparados

Se compararon:

- `LogisticRegression;`

- DecisionTreeClassifier;
- RandomForestClassifier.

11.6. Métrica principal

La métrica principal fue f1-score macro. Esta métrica es adecuada porque el problema tiene tres clases y puede existir desbalance entre bajo, medio y alto.

11.7. Elección de Random Forest

RandomForestClassifier fue seleccionado como modelo principal porque:

- es adecuado para datos tabulares;
- captura relaciones no lineales;
- ofrece buen desempeño sin requerir modelos pesados;
- es eficiente para una laptop con 16 GB de RAM;
- mantiene una complejidad razonable para fines académicos.

11.8. Resultados obtenidos

El modelo registrado fue RandomForestClassifier versión `v0.1.0`. El quality gate fue aprobado con f1-score macro superior al umbral mínimo configurado. Debe aclararse que las métricas son optimistas debido a que las etiquetas iniciales son generadas por reglas y algunas variables derivadas participan en el entrenamiento.

12. Pipeline ML

El pipeline ML se organiza en las siguientes etapas:

12.1. Ingesta de ubicaciones

El script `ingest\_locations.py` valida `data/external/inei\_puno\_districts.csv` y genera `data/processed/locations\_puno.csv`.

12.2. Ingesta de clima

El script `ingest\_weather\_open\_meteo.py` consulta Open-Meteo por coordenadas y guarda `data/raw/weather\_open\_meteo.csv`.

12.3. Construcción de features

El script `build\_features.py` une ubicaciones y clima, calcula variables temporales y genera etiquetas de riesgo.

12.4. Validación de datos

El script ``validate_dataset.py`` verifica columnas requeridas, valores nulos críticos, clases esperadas y rangos climáticos.

12.5. Entrenamiento

El script ``train_models.py`` entrena y compara modelos. Usa ``train_test_split`` con ``random_state`` fijo.

12.6. Evaluación

El script ``evaluate_model.py`` genera métricas y matriz de confusión.

12.7. Registry

El modelo y metadata se guardan en:

```
ml_pipeline/registry/frost_risk_model.joblib
ml_pipeline/registry/model_metadata.json
```

12.8. Quality gate

El script ``check_model_quality.py`` lee ``f1_macro`` desde metadata y falla si no supera el umbral mínimo.

13. Backend FastAPI

El backend FastAPI expone los siguientes endpoints:

13.1. GET /health

Permite verificar el estado del backend y disponibilidad del modelo registrado.

13.2. GET /ml/model-info

Devuelve información del modelo: nombre, versión, features, target, métricas, tamaño de dataset, fuentes y limitaciones.

13.3. POST /predict/frost-risk

Recibe un JSON con variables territoriales y climáticas. Valida la entrada con Pydantic, transforma el request en features, ejecuta el modelo y devuelve:

- nivel de riesgo;
- confianza;
- recomendación;
- condición para chuíño;
- versión del modelo;

- fuentes de datos.

13.4. GET /predictions/history

Devuelve historial desde Supabase si está habilitado. Si Supabase no está configurado, usa fallback NoOp y retorna lista vacía.

13.5. Pydantic, errores, CORS y repositorios

FastAPI usa Pydantic para validar contratos de entrada y salida. El manejo de errores evita exponer detalles internos del modelo. Se configuró CORS para permitir consumo desde Flutter Web local. La persistencia usa un patrón de repositorio con dos implementaciones:

- ``SupabasePredictionRepository`;`
- ``NoOpPredictionRepository`.`

14. Base de datos Supabase

Supabase se preparó con migraciones SQL, seed y políticas RLS.

14.1. Tablas

- ``users_profile``: perfiles de usuario.
- ``inei_locations``: ubicaciones territoriales.
- ``populated_centers``: centros poblados.
- ``agricultural_context``: contexto agrícola.
- ``weather_records``: registros meteorológicos.
- ``frost_predictions``: predicciones generadas.
- ``alerts``: alertas futuras.
- ``model_versions``: versiones del modelo.
- ``training_runs``: ejecuciones de entrenamiento.
- ``data_sources_log``: trazabilidad de fuentes de datos.

14.2. Seguridad y RLS

Se habilita Row Level Security en tablas del esquema público. Las tablas territoriales pueden leerse públicamente. Las predicciones se insertan desde backend usando ``service_role``, nunca desde Flutter. En el frontend no se almacenan credenciales de Supabase.

14.3. Service role

La clave ``service_role`` solo debe existir en el backend o en secretos seguros de infraestructura. No debe exponerse en Flutter, Flutter Web, repositorios ni variables públicas.

15. Aplicación Flutter

La app Flutter implementa una arquitectura modular por features:

- ``home``;
- ``prediction``;
- ``history``;
- ``data_sources``;
- ``model_info``;
- ``shell``.

15.1. Pantallas implementadas

- Home;
- PredictionForm;
- Result;
- History;
- DataSources;
- ModelInfo.

15.2. Consumo de API

Flutter consume únicamente FastAPI mediante ``ApiClient`` y ``FrostApiService``. No se comunica directamente con Supabase.

15.3. Diseño inspirado en Stitch

El diseño toma como referencia el concepto visual ``stitch_frost_puno_predictor``, usando una paleta con deep navy, ice blue, muted teal, soft white y amber para alertas.

15.4. Preparación para Web y móvil

La app fue creada para Android y Web. Usa ``--dart-define=API_BASE_URL`` para configurar el backend según entorno.

16. CI/CD y mejora continua

16.1. Workflows implementados

backend-tests.yml

Ejecuta pruebas del backend sin requerir Supabase real.

data-validation.yml

Valida ubicaciones, construye un dataset demo y verifica columnas, nulos y rangos.

ml-training.yml

Ejecuta pipeline ML completo con fechas configurables y guarda artefactos.

model-quality-gate.yml

Evalúa `model\_metadata.json` y falla si `f1\_macro` no supera el umbral mínimo.

16.2. Ciclo de mejora continua

```
datos nuevos -> validación -> entrenamiento -> evaluación -> quality gate ->
versionamiento -> despliegue controlado
```

Este flujo permite mantener el modelo bajo criterios mínimos de calidad antes de promover nuevas versiones.

17. Evidencias de implementación

Las siguientes capturas fueron generadas localmente mediante scripts de apoyo ubicados en `tools/screenshots/`. Estas evidencias documentan la estructura del proyecto, el pipeline de aprendizaje de máquina, el backend FastAPI, la aplicación Flutter Web, las pruebas y los componentes de CI/CD.

Captura 01: Estructura del proyecto

Captura 01. Estructura del proyecto

Estructura del proyecto

Arbol generado localmente con carpetas principales y archivos relevantes.

Frost Puno - estructura del proyecto
 Generado localmente para evidencias del informe.
 Raiz: frost-puno

```

|-- .github
|   |-- workflows
|       |-- backend-tests.yml
|       |-- data-validation.yml
|       |-- ml-training.yml
|       |-- model-quality-gate.yml
|-- app_flutter
|   |-- .idea
|       |-- libraries
|       |   |-- Dart_SDK.xml
|       |   |-- KotlinJavaRuntime.xml
|       |-- runConfigurations
|       |   |-- main_dart.xml
|       |-- modules.xml
|       |-- workspace.xml
|   |-- android
|       |-- app
|           |-- src
|           |   |-- debug
|           |   |-- main
|           |   |-- profile
|           |-- build.gradle.kts
|       |-- gradle
|           |-- wrapper
|               |-- gradle-wrapper.jar
|               |-- gradle-wrapper.properties
|       |-- .gitignore
|       |-- app_flutter_android.iml
|       |-- build.gradle.kts
|       |-- gradle.properties
|       |-- gradlew
|       |-- gradlew.bat
|       |-- local.properties
|       |-- settings.gradle.kts
|   |-- lib
|       |-- core
|           |-- api
|           |   |-- api_client.dart
|           |   |-- frost_api_service.dart
|           |-- config
|           |   |-- app_config.dart
|           |-- models
|           |   |-- health_status.dart
|           |-- theme
|           |   |-- app_colors.dart
|           |   |-- app_theme.dart
|       |-- features
|           |-- data_sources
|           |   |-- models
|           |   |-- screens
|           |-- history
|           |   |-- models
|           |   |-- screens
|           |-- home
|           |   |-- screens
|           |-- model_info
|           |   |-- models
|           |   |-- screens
|           |-- prediction
|           |   |-- models
|           |   |-- screens
|           |-- shell
|           |   |-- app_shell.dart
|       |-- shared
|           |-- widgets
|           |   |-- frost_app_bar.dart
|           |   |-- glass_card.dart
|           |   |-- page_scaffold.dart
|           |   |-- primary_action_button.dart
|           |   |-- responsive_content.dart
|           |   |-- section_header.dart
|           |   |-- status_chip.dart
|       |-- app.dart
|       |-- main.dart
|-- test
|   |-- widget_test.dart
|-- web
|   |-- icons
|   |   |-- Icon-192.png
|   |   |-- Icon-512.png
|   |   |-- Icon-maskable-192.png
|   |   |-- Icon-maskable-512.png
|   |-- favicon.png
|   |-- index.html
|   |-- manifest.json
|-- .gitignore
|-- .metadata
|-- analysis_options.yaml
|-- app_flutter.iml
|-- pubspec.lock
|-- pubspec.yaml
|-- README.md
|-- static_web.err
|-- static_web.log
-- backend_fastapi
|   |-- app
|       |-- api
|           |-- routes
|           |   |-- __init__.py
|           |   |-- health.py
|           |   |-- ml.py
|           |   |-- predict.py
|           |   |-- predictions.py
|           |   |-- __init__.py
|       |-- core
|           |-- __init__.py
|           |-- config.py
|           |-- exceptions.py
|       |-- repositories
|           |-- __init__.py
|           |-- predictions.py
|           |-- supabase_prediction_repository.py

```

La captura presenta la organización principal del repositorio Frost Puno. Se observa la separación entre `app\_flutter`, `backend\_fastapi`, `ml\_pipeline`, `data`, `docs`, `supabase` y `.github/workflows`, lo que evidencia una estructura modular por responsabilidades.

Captura 02: Dataset generado

Captura 02. Dataset generado

Frost Puno - evidencia local

Dataset generado

Primeras filas del dataset procesado para entrenamiento supervisado.

```
fecha,time,departamento,provincia,distrito,ubigeo,latitud,longitud,altitud_estimada,poblacion_total,poblacion_rural,porcentaje_rural,actividad_agropecuaria,cultivo_relevante,temperature_2m,relative_humidity_2m,apparent_temperature,dew_point_2m,precipitation,cloud_cover,wind_speed_10m,mes,hora,temporada_agricola,temperatura_minima_diaria,horas_bajo_cero,riesgo_helada
2024-06-01,2024-06-01T00:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,3.0,38,-1.1,-9.2,0.0,0.11,0.6,0,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T01:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,2.7,36,-2.3,-10.8,0.0,0.11,0.6,1,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T02:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,2.1,36,-3.1,-11.5,0.0,0.12,4.6,2,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T03:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,1.9,35,-3.3,-11.9,0.0,0.12,3.6,3,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T04:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,1.7,36,-3.4,-11.7,0.0,0.11,3.6,4,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T05:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,2.2,36,-2.4,-11.5,0.0,0.8,4.6,5,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T06:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,6.5,22,2.1,-14.0,0.0,0.5,9.6,6,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T07:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,5.0,40,1.2,-7.4,0.0,0.5,4.6,7,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T08:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,7.2,29,3.3,-9.5,0.0,0.4,9.6,8,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T09:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,10.0,19,6.0,-12.4,0.0,0.4,0.6,9,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T10:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,11.8,18,8.4,-11.6,0.0,0.7,4.6,10,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T11:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,13.2,19,10.3,-10.1,0.0,0.9,0.6,11,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T12:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,14.4,17,11.6,-10.5,0.0,0.10,0.6,12,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T13:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,15.2,15,12.3,-11.4,0.0,0.9,5.6,13,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T14:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,15.8,14,12.4,-12.1,0.0,0.8,7.6,14,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T15:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,15.8,15,11.1,-11.2,0.0,0.10,8.6,15,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T16:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,14.8,21,9.7,-7.3,0.0,0.13,8.6,16,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T17:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,13.0,29,8.2,-4.7,0.0,0.14,1.6,17,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T18:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,11.6,39,8.0,-1.9,0.0,0.8,5.6,18,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T19:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,10.5,47,7.5,-0.2,0.0,0.5,2.6,19,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T20:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,10.6,40,7.9,-2.4,0.0,0.1,5.6,20,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T21:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,7.3,58,4.1,-0.4,0.0,0.6,9.6,21,heladas_invernales,1.7,0,medio
2024-06-01,2024-06-01T22:00,Puno,Puno,Puno,210101,-15.8402,-70.0219,3827,128637,21868,16.99,True,papa,6.1,59,2.5,-1.4,0.0,0.8,6.6,22,heladas_invernales,1.7,0,medio
```

La evidencia muestra una vista del dataset procesado usado para entrenamiento. Este archivo consolida variables territoriales y climáticas, y sirve como base para la clasificación supervisada del riesgo de helada.

Captura 03: Metadata del modelo

Captura 03. Metadata del modelo

Model metadata

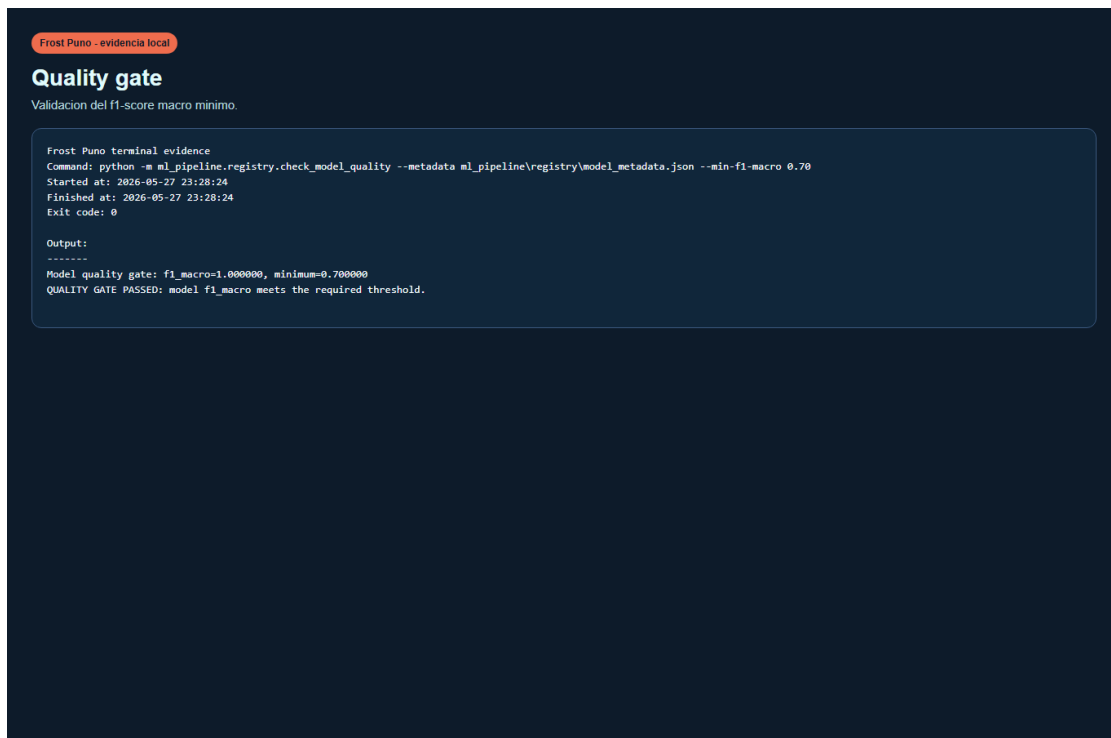
Registro del mejor modelo entrenado y sus métricas de evaluación.

```
{
  "model_name": "RandomForestClassifier",
  "version": "v0.1.0",
  "created_at": "2026-05-28T02:46:41+00:00",
  "features": [
    "latitud",
    "longitud",
    "altitud_estimada",
    "poblacion_total",
    "poblacion_rural",
    "porcentaje_rural",
    "temperature_2m",
    "relative_humidity_2m",
    "apparent_temperature",
    "dew_point_2m",
    "precipitation",
    "cloud_cover",
    "wind_speed_10m",
    "mes",
    "hora",
    "temperatura_minima_diaria",
    "horas_bajo_cero"
  ],
  "target": "riesgo_helada",
  "metrics": {
    "accuracy": 1,
    "precision_macro": 1,
    "recall_macro": 1,
    "f1_macro": 1
  },
  "all_model_metrics": {
    "LogisticRegression": {
      "accuracy": 0.9963369963369964,
      "precision_macro": 0.9965095986038395,
      "recall_macro": 0.9968701095461658,
      "f1_macro": 0.9966732869910625
    },
    "DecisionTreeClassifier": {
      "accuracy": 1,
      "precision_macro": 1,
      "recall_macro": 1,
      "f1_macro": 1
    },
    "RandomForestClassifier": {
      "accuracy": 1,
      "precision_macro": 1,
      "recall_macro": 1,
      "f1_macro": 1
    }
  },
  "dataset_size": 4368,
  "data_sources": [
    "Open-Meteo Historical API",
    "INEI-compatible curated district seed for Puno MVP",
    "SENAMPHI documented for validation in phase 2",
    "MIDAGRI/SIEA documented as agrarian enrichment in phase 2"
  ],
  "limitations": [
    "Initial labels are rule-based and derived from temperature thresholds.",
    "The curated INEI district file is an MVP seed and must be replaced with official exports for production.",
    "Daily minimum temperature and hours below zero are label-derived features, so baseline metrics can be optimistic.",
    "SENAMPHI station observations are not yet used as ground-truth labels."
  ]
}
```

La captura muestra el archivo `model\_metadata.json` generado por el registry del pipeline ML. Incluye versión del modelo, algoritmo seleccionado, métricas, tamaño del dataset, fuentes de datos y limitaciones documentadas.

Captura 04: Quality gate aprobado

Captura 04. Quality gate aprobado



La evidencia muestra la validación del umbral mínimo de `f1-score macro`. Este control impide promover un modelo si no cumple el criterio básico de calidad definido para el MVP académico.

Captura 05: FastAPI Swagger

Captura 05. FastAPI Swagger

Frost Puno API 0.1.0 OAS 3.1

/openapi.json

Backend MVP for Frost Puno frost-risk prediction.

health

GET

/health Health Check

ml

GET

/ml/model-info Model Info

prediction

POST

/predict/frost-risk Predict Frost Risk

predictions

GET

/predictions/history Prediction History

Schemas

FrostPredictionHistoryItem > Expand all object

FrostRiskRequest > Expand all object

FrostRiskResponse > Expand all object

HTTPValidationError > Expand all object

HealthResponse > Expand all object

ModelInfoResponse > Expand all object

ValidationError > Expand all object

La captura presenta la documentación automática de FastAPI mediante Swagger UI. Se visualizan los endpoints principales del backend, lo cual facilita la validación técnica y la exposición controlada de la API.

Captura 06: Endpoint de salud

Captura 06. Endpoint de salud

Pretty-print ☐

```
("status":"ok", "app_name":"Frost Puno API", "version":"0.1.0", "model_available":true)
```

Esta evidencia muestra la respuesta de `GET /health`. El endpoint permite comprobar que el backend se encuentra operativo y que el modelo está disponible para atender predicciones.

Captura 07: Endpoint de información del modelo

Captura 07. Endpoint de información del modelo

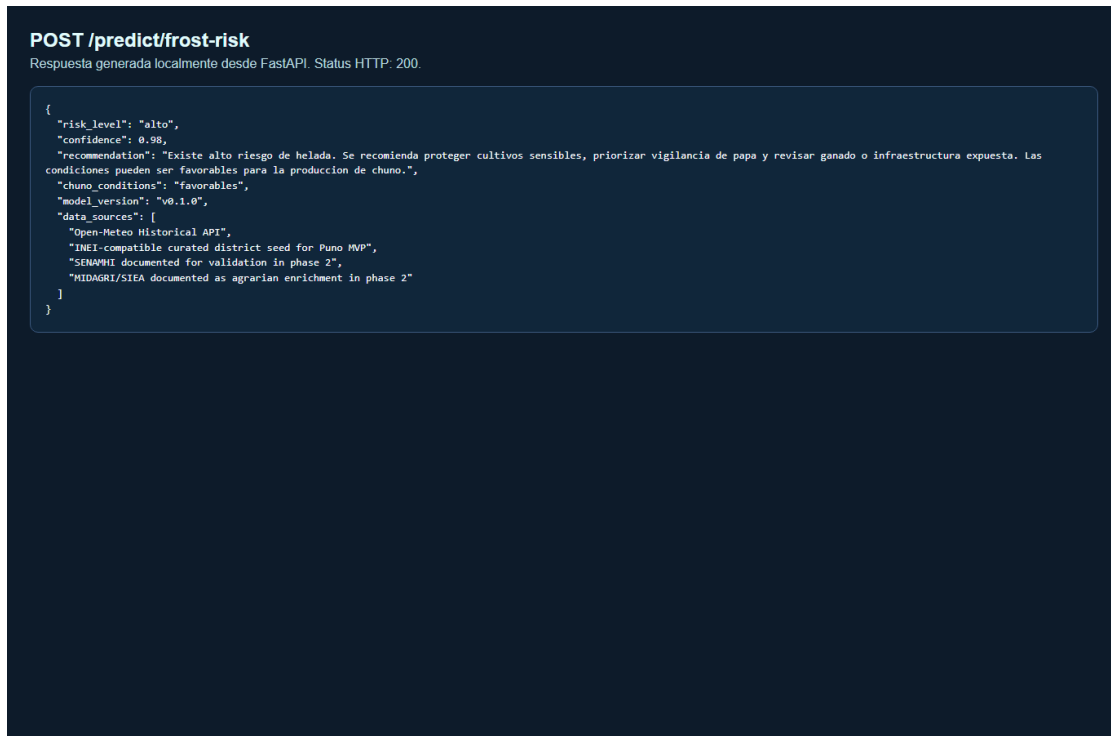
Pretty-print ☐

```
("model_name":"RandomForestClassifier", "version":"v0.1.0", "created_at":"2026-05-28T02:46:41+00:00", "features":  
["latitud", "longitud", "altitud_estimada", "poblacion_total", "poblacion_rural", "porcentaje_rural", "temperature_2m", "relative_humidity_2m", "apparent_temperature", "dew_point_2m",  
precipitation", "cloud_cover", "wind_speed_10m", "mes", "hora", "temperatura_minima_diaria", "horas_bajo_cero"], "target":"riesgo_helada", "metrics":  
{"accuracy":1.0, "precision_macro":1.0, "recall_macro":1.0, "f1_macro":1.0}, "dataset_size":4368, "data_sources":["Open-Meteo Historical API", "INEI-compatible curated district seed  
for Puno MVP", "SENAMHI documented for validation in phase 2", "MIDAGRI/SIDA documented as agrarian enrichment in phase 2"], "limitations":["Initial labels are rule-based and  
derived from temperature thresholds.", "The curated INEI district file is an MVP seed and must be replaced with official exports for production.", "Daily minimum temperature and  
hours below zero are label-derived features, so baseline metrics can be optimistic.", "SENAMHI station observations are not yet used as ground-truth labels."])
```

La captura muestra la respuesta de `GET /ml/model-info`. Este endpoint expone información técnica del modelo activo, su versión, métricas y fuentes de datos usadas.

Captura 08: Endpoint de predicción

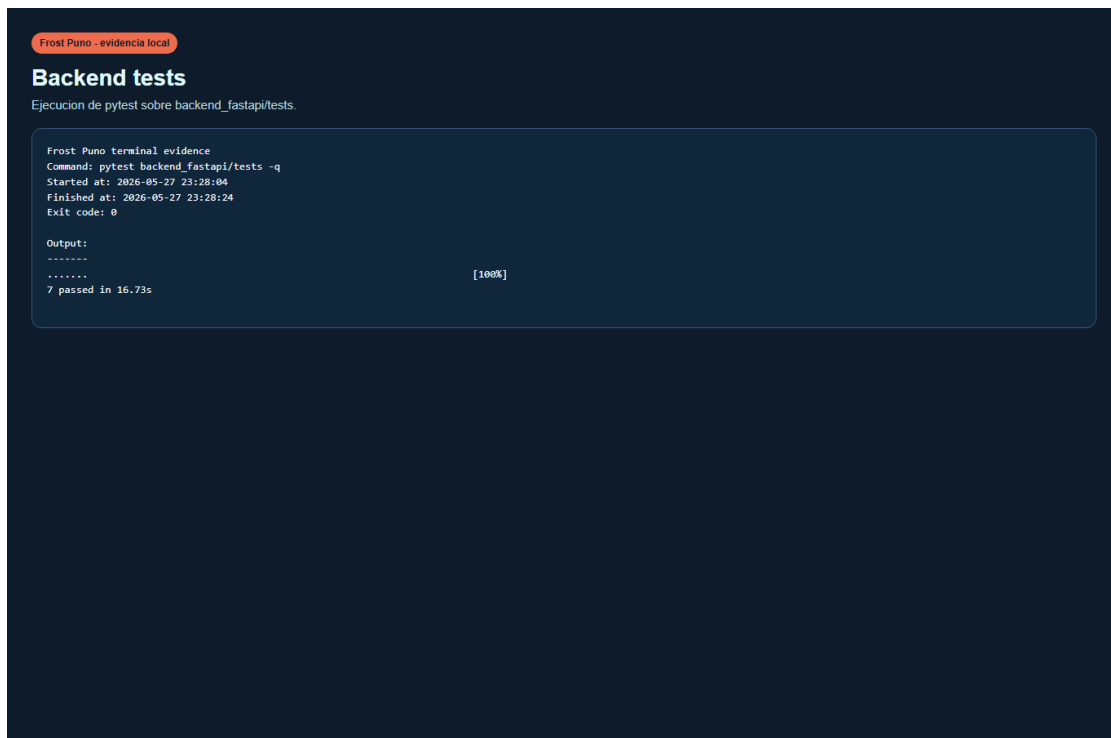
Captura 08. Endpoint de predicción



La evidencia muestra una respuesta JSON generada por `POST /predict/frost-risk`. Se observa el nivel de riesgo, la confianza, la recomendación, la condición para chuño, la versión del modelo y las fuentes asociadas.

Captura 09: Tests backend

Captura 09. Tests backend



La captura documenta la ejecución de pruebas automatizadas del backend. Estas pruebas verifican endpoints principales, predicción, historial sin Supabase y comportamiento del repositorio NoOp.

Captura 10: Flutter Home

Captura 10. Flutter Home

● Sistema activo

Frost Puno

Prediccion inteligente de heladas para comunidades altoandinas.

Riesgo actual de helada ❄️

-4.2 °C

💧 Hum 42%

🌬️ 12 km/h

→ Consultar riesgo



Fuentes de

La evidencia presenta la pantalla inicial de Flutter Web. Se aprecia el estado del sistema, la tarjeta de riesgo actual, accesos rápidos y la estética visual inspirada en el diseño de Stitch.

Captura 11: Formulario Flutter

Captura 11. Formulario Flutter



Consulta de riesgo de helada

Ingrese los parametros territoriales y climaticos para ejecutar el modelo predictivo.



Parametros Territoriales

DISTRITO

Puno

PROVINCIA

Puno

CENTRO POBLADO

Centro poblado demo

LATITUD

-15.8402

LONGITUD

-70.0219

ALTITUD MSNM

3827

La captura muestra el formulario de consulta con datos demo precargados. Esta pantalla permite enviar parámetros territoriales y climáticos al backend FastAPI sin exponer credenciales de Supabase.

Captura 12: Resultado de predicción

Captura 12. Resultado de predicción



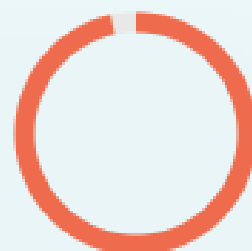
Resultado de prediccion

Lectura del modelo para la ubicacion y condiciones enviadas.

RIESGO



Alto



CONFIANZA

97%

CONDICION PARA
CHUNO



Favorable

Existe alto riesgo de helada. Se recomienda proteger cultivos sensibles, priorizar vigilancia de papa y revisar ganado o infraestructura expuesta. Las condiciones pueden ser favorables para la produccion de

La evidencia muestra el resultado de predicción retornado por el backend. La interfaz comunica el nivel de riesgo, la confianza del modelo y una recomendación interpretativa para el usuario.

Captura 13: Historial

Captura 13. Historial

Historial de predicciones

Predicciones registradas por FastAPI cuando Supabase esta activo.

 Filtrar por distrito...



Aun no hay predicciones guardadas.
Con Supabase apagado, FastAPI
devuelve una lista vacia.

La captura presenta la pantalla de historial de predicciones. En modo MVP puede mostrarse una lista vacía si Supabase está desactivado, manteniendo el flujo preparado para persistencia real.

Captura 14: Fuentes de datos

Captura 14. Fuentes de datos

Fuentes de datos

Referencias y orígenes de la información analizada por el sistema.

Open-Meteo

CLIMA

Fuente climática principal del MVP.
Provee datos meteorológicos históricos y pronósticos por coordenadas.

INEI

TERRITORIO

Fuente territorial, censal y agropecuaria para ubigeos, distritos, ruralidad y contexto local.



SENAMHI

La evidencia muestra la pantalla de fuentes consideradas por el sistema. Se documenta el rol de Open-Meteo, INEI, SENAMHI y MIDAGRI/SIEA dentro de la arquitectura del MVP y fases futuras.

Captura 15: Información del modelo en Flutter

Captura 15. Información del modelo



Informacion del modelo

Especificaciones tecnicas y metricas de rendimiento del clasificador climatico actual.

ARQUITECTURA BASE

RandomForestClassifier



Ensemble Learning

Scikit-Learn



METRICA PRINCIPAL

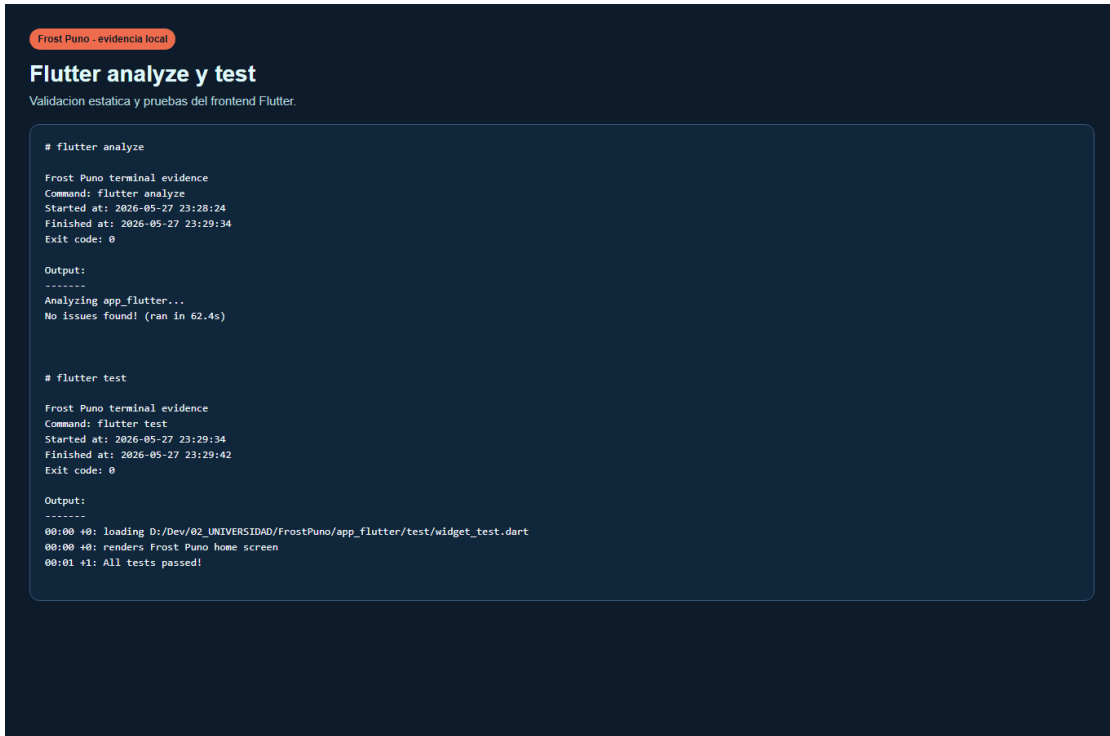
1.00 F1 macro

Balance entre precision y exhaustividad para tres clases de riesgo.

La captura muestra la pantalla Flutter que consume `GET /ml/model-info`. Permite evidenciar que el frontend obtiene información técnica del modelo únicamente a través del backend FastAPI.

Captura 16: Flutter analyze y test

Captura 16. Flutter analyze y test

A screenshot of a terminal window with a dark blue background. At the top left, there is a small orange pill-shaped badge with the text "Frost Puno - evidencia local". Below it, the title "Flutter analyze y test" is displayed in white, followed by the subtitle "Validación estática y pruebas del frontend Flutter." in a smaller font. The main content of the terminal is a log of two commands: "flutter analyze" and "flutter test". The "flutter analyze" section shows the command being executed, the start and end times (2026-05-27 23:28:24 to 23:29:34), the exit code (0), and the output "Analyzing app_flutter... No issues found! (ran in 62.4s)". The "flutter test" section shows the command being executed, the start and end times (2026-05-27 23:29:34 to 23:29:42), the exit code (0), and the output "00:00 00: loading D:/Dev/02_UNIVERSIDAD/FrostPuno/app_flutter/test/widget_test.dart", "00:00 00: renders Frost Puno home screen", and "00:01 01: All tests passed!".

```
# flutter analyze

Frost Puno terminal evidence
Command: flutter analyze
Started at: 2026-05-27 23:28:24
Finished at: 2026-05-27 23:29:34
Exit code: 0

Output:
-----
Analyzing app_flutter...
No issues found! (ran in 62.4s)

# flutter test

Frost Puno terminal evidence
Command: flutter test
Started at: 2026-05-27 23:29:34
Finished at: 2026-05-27 23:29:42
Exit code: 0

Output:
-----
00:00 00: loading D:/Dev/02_UNIVERSIDAD/FrostPuno/app_flutter/test/widget_test.dart
00:00 00: renders Frost Puno home screen
00:01 01: All tests passed!
```

La evidencia registra la validación estática y pruebas del proyecto Flutter. Estos comandos respaldan que la interfaz se mantiene sin errores de análisis y con pruebas funcionales para el MVP.

Captura 17: Flutter build web

Captura 17. Flutter build web

```
Frost Puno terminal evidence
Command: flutter build web --dart-define=API_BASE_URL=http://127.0.0.1:8000
Started at: 2026-05-27 23:29:42
Finished at: 2026-05-27 23:30:40
Exit code: 0

Output:
-----
Compiling lib/main.dart for the Web...
flutter : Wasm dry run succeeded. Consider building and testing your application with the '--wasm' flag. See docs for
more info: https://docs.flutter.dev/platform-integration/web/wasm
En D:\Dev\02_UNIVERSIDAD\FrostPuno\tools\screenshots\generate_terminal_evidence.ps1: 89 Carácter: 16
+ ... -Command { flutter build web --dart-define=API_BASE_URL=http://127.0 ...
+
+ CategoryInfo          : NotSpecified: (Wasm dry run su...ration/web/wasm:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

Use --no-wasm-dry-run to disable these warnings.
Compiling lib/main.dart for the Web... 54.7s
✓ Built build/web
```

La captura muestra la compilación correcta de Flutter Web. Esto demuestra que el frontend está preparado para una demo local y para un despliegue posterior en una plataforma gratuita compatible.

Captura 18: GitHub workflows

Captura 18. GitHub workflows

GitHub Actions workflows

Workflows implementados para backend, datos, entrenamiento y quality gate.

```
# .github/workflows/backend-tests.yml

name: Backend Tests

on:
  push:
  pull_request:

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    env:
      ENABLE_SUPABASE: "false"
      PYTHONPATH: ${GITHUB_WORKSPACE}/backend_fastapi

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: |
            backend_fastapi/requirements.txt
            ml_pipeline/requirements.txt

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r backend_fastapi/requirements.txt
          pip install -r ml_pipeline/requirements.txt

      - name: Run backend tests
        run: pytest backend_fastapi/tests -q

# .github/workflows/data-validation.yml

name: Data Validation

on:
  push:
  pull_request:
  workflow_dispatch:

jobs:
  data-validation:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
          cache-dependency-path: ml_pipeline/requirements.txt

      - name: Install ML dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r ml_pipeline/requirements.txt

      - name: Validate and process locations
        run: python -m ml_pipeline.data_ingestion.ingest_locations

      - name: Build demo weather dataset
        run: >
          python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo
          --start-date 2024-06-01
          --end-date 2024-06-03
          --limit 3
          --max-workers 3

      - name: Build features
        run: python -m ml_pipeline.features.build_features

      - name: Validate training dataset
        run: python -m ml_pipeline.preprocessing.validate_dataset

# .github/workflows/ml-training.yml

name: ML Training

on:
  workflow_dispatch:
  inputs:
    start_date:
      description: "Open-Meteo historical start date"
      required: false
      default: "2024-06-01"
    end_date:
      description: "Open-Meteo historical end date"
      required: false
      default: "2024-06-07"
    location_limit:
      description: "Number of locations for demo training. Use empty for all curated locations."
      required: false
      default: "5"
    model_version:
      description: "Model version label"
      required: false
      default: "ci-demo"

  schedule:
    - cron: "0 9 * * 1"

jobs:
  train-model:
    runs-on: ubuntu-latest
```

La evidencia muestra los workflows de GitHub Actions implementados. Estos archivos sustentan la automatización de pruebas, validación de datos, entrenamiento del modelo y control de calidad.

Captura 19: Supabase SQL

Captura 19. Supabase SQL

Supabase SQL

Migraciones, políticas RLS y datos semilla preparados para el MVP.

```
# supabase/migrations/001_create_core_tables.sql

create extension if not exists pgcrypto;

create or replace function public.set_updated_at()
returns trigger
language plpgsql
as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

create table if not exists public.users_profile (
  id uuid primary key default gen_random_uuid(),
  user_id uuid unique references auth.users(id) on delete cascade,
  full_name text,
  role text not null default 'producer' check (role in ('producer', 'student', 'admin', 'researcher')),
  department text not null default 'Puno',
  province text,
  district text,
  community text,
  source text not null default 'app',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table if not exists public.inei_locations (
  id uuid primary key default gen_random_uuid(),
  ubigeo text not null unique check (ubigeo ~ '^[0-9]{6}$'),
  department text not null,
  province text not null,
  district text not null,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  estimated_altitude numeric(8,2) check (estimated_altitude between 0 and 7000),
  total_population integer check (total_population >= 0),
  rural_population integer check (rural_population >= 0),
  rural_percentage numeric(5,2) check (rural_percentage between 0 and 100),
  source text not null default 'INEI',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now(),
  constraint rural_population_lte_total check (
    total_population is null
    or rural_population is null
    or rural_population <= total_population
  )
);

create table if not exists public.populated_centers (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete set null,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
  name text not null,
  category text,
  latitude numeric(9,6) check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) check (longitude between -71.5 and -68.0),
  altitude numeric(8,2) check (altitude between 0 and 7000),
  population integer check (population >= 0),
  source text not null default 'INEI Sistema de Centros Poblados',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table if not exists public.agricultural_context (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete cascade,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete cascade,
  crop text not null,
  agricultural_activity boolean not null default true,
  cultivated_area_ha numeric(12,2) check (cultivated_area_ha is null or cultivated_area_ha >= 0),
  production_tons numeric(12,2) check (production_tons is null or production_tons >= 0),
  yield_ton_ha numeric(12,4) check (yield_ton_ha is null or yield_ton_ha >= 0),
  reference_year integer check (reference_year between 1900 and 2100),
  source text not null default 'INEI/MIDAGRI-SIEA',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table if not exists public.weather_records (
  id uuid primary key default gen_random_uuid(),
  location_id uuid references public.inei_locations(id) on delete set null,
  ubigeo text references public.inei_locations(ubigeo) on update cascade on delete set null,
  observed_at timestampz not null,
  latitude numeric(9,6) not null check (latitude between -18.5 and -13.0),
  longitude numeric(9,6) not null check (longitude between -71.5 and -68.0),
  temperature_2m numeric(6,2),
  relative_humidity_2m numeric(5,2) check (relative_humidity_2m between 0 and 100),
  apparent_temperature numeric(6,2),
  dew_point_2m numeric(6,2),
  precipitation numeric(8,2) check (precipitation is null or precipitation >= 0),
  cloud_cover numeric(5,2) check (cloud_cover between 0 and 100),
  wind_speed_10m numeric(7,2) check (wind_speed_10m is null or wind_speed_10m >= 0),
  source text not null default 'Open-Meteo',
  metadata jsonb not null default '{}'::jsonb,
  created_at timestampz not null default now()
);

create table if not exists public.model_versions (
  id uuid primary key default gen_random_uuid(),
  version text not null unique,
  model_name text not null,
  algorithm text not null,
  artifact_path text not null,
  accuracy numeric(8,6) check (accuracy between 0 and 1),
  precision_macro numeric(8,6) check (precision_macro between 0 and 1),
  recall_macro numeric(8,6) check (recall_macro between 0 and 1),
  f1_macro numeric(8,6) check (f1_macro between 0 and 1),
  dataset_size integer not null check (dataset_size >= 0),
  status text not null default 'candidate' check (status in ('candidate', 'active', 'rejected', 'archived')),
  source text not null default 'ml_pipeline',

```

La captura presenta las migraciones, políticas RLS y datos semilla de Supabase. Esta evidencia respalda el diseño de persistencia, seguridad básica y preparación de la base de datos del MVP.

18. Resultados obtenidos

Los resultados del MVP son:

- 13 distritos procesados.
- 4,368 filas horarias climáticas generadas desde Open-Meteo.
- Distribución de clases:
 - alto = 1704;
 - medio = 1512;
 - bajo = 1152.
- Mejor modelo registrado: RandomForestClassifier.
- Quality gate aprobado.
- Backend tests: 7 passed.
- Flutter analyze: sin errores.
- Flutter test: passed.
- Flutter build web: correcto.
- Backend FastAPI funcional con endpoints mínimos.
- Supabase preparado con migraciones, seed y RLS.
- Aplicación Flutter implementada con diseño inspirado en Stitch.

19. Limitaciones

22. INEI se usa mediante una semilla curada MVP, no mediante extracción oficial completa automática.
23. SENAMHI aún no está integrado como validación oficial completa.
24. Las etiquetas iniciales se generan con reglas térmicas, no con observaciones de daño agrícola.
25. El dataset inicial tiene cobertura temporal y territorial limitada.
26. Las métricas pueden ser optimistas por el uso de variables derivadas de la regla de etiquetado.
27. Render, Vercel y Supabase free tier tienen límites de disponibilidad, almacenamiento y ejecución.
28. El MVP no reemplaza sistemas oficiales de alerta meteorológica ni recomendaciones técnicas institucionales.

20. Trabajos futuros

29. Integrar datos SENAMHI de estaciones y avisos de helada.

30. Reemplazar la semilla INEI por exportaciones oficiales completas.
31. Integrar MIDAGRI/SIEA para cultivos, superficie, producción y rendimiento.
32. Implementar mapa interactivo por distrito o centro poblado.
33. Agregar alertas push con Firebase Cloud Messaging.
34. Crear dashboard administrativo.
35. Implementar modo offline para comunidades con conectividad limitada.
36. Desplegar completamente en Vercel, Render y Supabase.
37. Validar predicciones con productores locales y especialistas.
38. Incorporar monitoreo de deriva de datos y comparación automática con modelo activo anterior.

21. Conclusiones

39. Frost Puno demuestra la viabilidad de integrar datos abiertos, aprendizaje supervisado y arquitectura distribuida para estimar riesgo de heladas en un contexto regional altoandino.
40. Desde Aprendizaje de Máquina, el proyecto implementa un ciclo completo: dataset, features, entrenamiento, evaluación, registro de modelo y quality gate.
41. Desde Computación Paralela y Distribuida, el sistema evidencia procesamiento concurrente por ubicaciones, separación modular de responsabilidades y automatización mediante jobs independientes.
42. El uso de RandomForestClassifier resulta adecuado para el MVP por su desempeño en datos tabulares y bajo costo computacional.
43. La arquitectura modular monolítica del backend es una decisión apropiada para un proyecto universitario, ya que reduce complejidad sin impedir escalabilidad futura.
44. La aplicación Flutter permite mostrar el sistema de manera visual, clara y preparada para web/móvil, sin exponer credenciales sensibles.
45. Las limitaciones del dataset y las etiquetas iniciales deben ser reconocidas explícitamente; el valor académico del sistema está en su arquitectura, trazabilidad y capacidad de mejora.

22. Bibliografía

FastAPI. (2026). *FastAPI documentation*. <https://fastapi.tiangolo.com/>

Flutter. (2026). *Build and release a web app*. <https://docs.flutter.dev/deployment/web>

GitHub. (2026). *Workflow syntax for GitHub Actions*. <https://docs.github.com/actions/reference/workflows-and-actions/workflow-syntax>

Google. (2026). *Stitch*. <https://stitch.withgoogle.com/>

Nota: usado como referencia conceptual de diseño visual cuando corresponda.

Instituto Nacional de Estadística e Informática. (2025). *Consultar datos en el Sistema de Información Distrital del INEI*. Plataforma del Estado Peruano. <https://www.gob.pe/institucion/inei/pages/27392-consultar-datos-en-el-sistema-de-informacion-distrital-del-inei>

Ministerio de Desarrollo Agrario y Riego. (2026). *Compendio anual de Producción Agrícola*. Plataforma del Estado Peruano. <https://www.gob.pe/institucion/midagri/informes-publicaciones/2730325-compendio-anual-de-produccion-agricola>

Open-Meteo. (2026). *Historical Weather API*. <https://open-meteo.com/en/docs/historical-weather-api>

Scikit-learn. (2026). *sklearn.ensemble: RandomForestClassifier*. <https://scikit-learn.org/stable/api/sklearn.ensemble.html>

Servicio Nacional de Meteorología e Hidrología del Perú. (2026). *Descarga de datos*. <https://www.senamhi.gob.pe/site/descarga-datos/?p=sedes>

Supabase. (2026). *Row Level Security*. <https://supabase.com/docs/guides/database/postgres/row-level-security>

Supabase. (2026). *Securing your API*. <https://supabase.com/docs/guides/api/securing-your-api>

23. Anexos

23.1. Comandos de ejecución

Backend FastAPI

```
Set-Location "D:\Dev\02_UNIVERSIDAD\FrostPuno"
$env:PYTHONPATH = "D:\Dev\02_UNIVERSIDAD\FrostPuno\backend_fastapi"
uvicorn app.main:app --reload --app-dir backend_fastapi
```

Flutter Web

```
Set-Location "D:\Dev\02_UNIVERSIDAD\FrostPuno\app_flutter"
flutter run -d chrome --dart-define=API_BASE_URL=http://127.0.0.1:8000
```

Pipeline ML

```
python -m ml_pipeline.data_ingestion.ingest_locations
python -m ml_pipeline.data_ingestion.ingest_weather_open_meteo --start-date 2024-06-01 --end-date 2024-06-14 --max-workers 4
python -m ml_pipeline.features.build_features
python -m ml_pipeline.preprocessing.validate_dataset
python -m ml_pipeline.training.train_models --version v0.1.0
python -m ml_pipeline.evaluation.evaluate_model
python -m ml_pipeline.registry.check_model_quality --metadata ml_pipeline\registry\model_metadata.json --min-f1-macro 0.70
```


23.2. Estructura de carpetas

```
FrostPuno/  
  app_flutter/  
  backend_fastapi/  
  data/  
  docs/  
  ml_pipeline/  
  supabase/  
  .github/workflows/
```

23.3. Ejemplo JSON de predicción

```
{  
  "district": "Puno",  
  "province": "Puno",  
  "populated_center": "Centro poblado demo",  
  "latitude": -15.8402,  
  "longitude": -70.0219,  
  "altitude": 3827,  
  "rural_population": 1200,  
  "agricultural_activity": true,  
  "main_crop": "papa",  
  "temperature_min": -2.5,  
  "temperature_max": 12.4,  
  "feels_like": -4.0,  
  "humidity": 68,  
  "wind_speed": 7,  
  "cloud_cover": 20,  
  "dew_point": -3.5,  
  "precipitation": 0,  
  "month": 6,  
  "hour": 3,  
  "hours_below_zero": 4  
}
```

23.4. Checklist de despliegue futuro

- [] Crear proyecto Supabase.
- [] Ejecutar migraciones SQL.
- [] Configurar RLS y seed.
- [] Configurar variables backend en Render.
- [] Configurar API URL en Flutter Web.
- [] Desplegar Flutter Web en Vercel.
- [] Configurar GitHub Actions.
- [] Ejecutar quality gate.
- [] Validar Swagger en entorno remoto.
- [] Validar predicción desde Flutter Web.
- [] Verificar que no existan claves Supabase en frontend.